

BỘ XÂY DỰNG
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN ĐÀO TẠO CHẤT LƯỢNG CAO



BÁO CÁO MÔN LẬP TRÌNH MẠNG

Đề tài:
Xây dựng ứng dụng Chat Peer-to-Peer (P2P)

Giảng viên :	Mai Ngọc Châu	
Sinh viên thực hiện:	MSSV:	Chức vụ:
1. Nguyễn Thanh Khánh	040206012006	Thành viên
2. Huỳnh Văn Tại	054206000426	Nhóm trưởng
3. Ngô Đặng Minh Khôi	089206018408	Thành viên
4. Ngủ Hoàng Khang	089206018080	Thành viên
5. Trần Ngô Duy Khang	082206013072	Thành viên
6. Lương Quốc Khánh	051206012692	Thành viên
Lớp:	012012301302	

TP. Hồ Chí Minh, ngày 25 tháng 09 năm 2026

LỜI NÓI ĐẦU

Kính gửi Thầy Mai Ngọc Châu,

Dưới đây là báo cáo tóm tắt nội dung đồ án "Xây dựng ứng dụng nhắn tin P2PChat (UDM09)" do Nhóm 10 thực hiện. Báo cáo này trình bày tổng quan về hệ thống, cơ sở lý thuyết, quá trình phân công nhiệm vụ chi tiết của từng thành viên và những bài học nhóm đã đúc kết được.

MỤC LỤC

LỜI NÓI ĐẦU.....	2
TỔNG QUAN DỰ ÁN.....	5
1. Lý Do Chọn Đề Tài	5
2. Mục Tiêu Đề Tài.....	6
2.1. Mục tiêu tổng quát	6
2.2. Mục tiêu cụ thể.....	6
CƠ SỞ LÝ THUYẾT	7
1. Mô hình Mạng Peer-to-Peer (Pure P2P).....	7
2. Lập trình Socket TCP và Giao thức UDP Broadcast.....	7
3. Kỹ thuật Framing Length-Prefix	7
4. Mô hình Mã hóa Lai (Hybrid Cryptography).....	7
5. Mô hình Quản lý Độ tin cậy TOFU (Trust On First Use)	7
KIẾN TRÚC VÀ CẤU TRÚC MÃ NGUỒN	8
CHI TIẾT LUỒNG HOẠT ĐỘNG CÁC CHỨC NĂNG.....	9
1. Luồng Khởi động & UDP Discovery	9
2. Luồng Thiết lập Kết nối P2P & Handshake	9
3. Luồng Gửi/Nhận Tin nhắn & Chồng treo GUI	9
4. Luồng Truyền tải Tập tin (File Transfer).....	9
THÀNH QUẢ ĐẠT ĐƯỢC.....	10
BẢNG PHÂN CÔNG CÔNG VIỆC CHI TIẾT	11
KHÓ KHĂN GẶP PHẢI.....	12
1. Sự cố dính gói và xé gói trên TCP Stream	12
2. Xung đột luồng làm đơ và crash giao diện Tkinter	12
3. Bảng thông bị nghẽn do chuỗi ảnh Avatar quá lớn	12
4. Dò tìm Peer (UDP Discovery) thất bại do Tường lửa và Wi-Fi.....	13
KIẾN THỨC / KỸ NĂNG HỌC ĐƯỢC	13
NỘI DUNG ĐÃ VÀ CHƯA HOÀN THÀNH	13
1. Nội dung đã hoàn thành (100%).....	13
2. Nội dung chưa hoàn thành / Hạn chế	14
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	14

THÔNG TIN REPOSITORY16

TỔNG QUAN DỰ ÁN

Thông tin chính:

Thông tin dự án	Chi tiết
Môn học	Lập trình mạng
Lớp	012012301302
Nhóm	Net3_Group_10 - UDM09
Ngôn ngữ	Python (Tkinter/CustomTkinter)
Mô hình	Peer-to-Peer (Pure P2P)
Repository	GitHub Link

1. Lý Do Chọn Đề Tài

Sự phụ thuộc vào các nền tảng nhắn tin phổ biến như Zalo, Messenger hay Telegram hiện nay đặt ra nhiều bài toán phức tạp về tính sẵn sàng và tính riêng tư. Thực tế cho thấy, việc toàn bộ dữ liệu người dùng phải đi qua một máy chủ trung tâm (Client-Server) khiến hệ thống dễ rơi vào trạng thái "tê liệt" hoàn toàn mỗi khi máy chủ gặp sự cố kỹ thuật hoặc bị tấn công từ chối dịch vụ (DDoS). Bên cạnh đó, những lo ngại về việc thông tin cá nhân bị thu thập, phân tích dữ liệu ngầm hay sự cố rò rỉ dữ liệu từ bên thứ ba ngày càng trở nên nghiêm trọng. Việc các doanh nghiệp phải chi trả khoản ngân sách khổng lồ để duy trì và mở rộng hạ tầng Server cũng là một điểm hạn chế lớn của mô hình này.

Mô hình mạng ngang hàng (Peer-to-Peer - P2P) mở ra một hướng đi khác: cho phép các thiết bị đầu cuối tự kết nối và trao đổi dữ liệu trực tiếp với nhau mà không qua bất kỳ máy chủ trung gian nào. Xuất phát từ nhu cầu thực tế đó, nhóm lựa chọn đề tài "Ứng dụng Chat P2P không dùng Server trung tâm (UDM09)" nhằm tự xây dựng một công cụ truyền thông nội bộ độc lập, bảo mật và hoàn toàn tự chủ về mặt hạ tầng.

Về mặt chuyên môn, đề tài này là cơ hội để nhóm sinh viên giải quyết trực tiếp các bài toán kỹ thuật thực tế trong môn Lập trình mạng thay vì chỉ dừng lại ở các bài tập ví dụ đơn giản:

- Lập trình Socket cấp thấp (TCP/IP & UDP): Tự tay quản lý vòng đời kết nối, khởi tạo Socket lắng nghe (Server) và Socket chủ động kết nối (Client) ngay trên cùng một nút mạng (Node).
- Xử lý định gói và xé gói (Framing Length-Prefix): Bản chất của luồng byte TCP là không có ranh giới gói tin.
- Đa luồng & An toàn luồng (Multithreading & Thread-Safety): Xử lý việc nhận dữ liệu từ các luồng mạng ngầm (Background Threads) và đẩy an toàn về vòng lặp sự kiện đơn của giao diện GUI.
- Bảo mật và Mô hình xác thực TOFU: Kết hợp thuật toán mã hóa bất đối xứng RSA-2048 để bắt tay trao đổi khóa, Fernet/AES-256-GCM để mã hóa tin nhắn thời gian thực

2. Mục Tiêu Đề Tài

2.1. Mục tiêu tổng quát

Xây dựng ứng dụng trò chuyện trực tiếp giữa các nút mạng (Peers) không qua máy chủ trung gian, hoạt động ổn định trên môi trường mạng nội bộ (LAN) với giao diện đồ họa GUI hoàn chỉnh.

2.2. Mục tiêu cụ thể

- Lập trình kết nối TCP P2P giữa các máy tính thông qua IP và Port.
- Phát hiện tự động các Peer đang hoạt động trong mạng LAN qua giao thức UDP Broadcast (Port 15000).
- Thiết kế gói tin JSON với tiền tố độ dài Length-Prefix 4-byte Big-Endian.
- Thực hiện Handshake trao đổi khóa công khai RSA-2048 và tạo khóa phiên mã hóa đối xứng Fernet / AES-256-GCM.
- Xây dựng mô hình quản lý độ tin cậy TOFU (trạng thái TRUSTED, VERIFIED, MISMATCH, BLOCKED).
- Hỗ trợ truyền file an toàn theo phân đoạn (Chunk 64KB), kiểm tra toàn vẹn SHA-256, giới hạn file 10MB.

- Đảm bảo giao diện GUI mượt mà, chống treo ứng dụng bằng cách xử lý đa luồng ngầm (Background Threads) và giới hạn hàng đợi render (deque maxlen=500, batch=25).

CƠ SỞ LÝ THUYẾT

1. Mô hình Mạng Peer-to-Peer (Pure P2P)

Mỗi máy tham gia mạng (Node) đóng vai trò bình đẳng: vừa làm TCP Server (lắng nghe kết nối tại Port chỉ định), vừa làm TCP Client (khởi tạo kết nối tới Peer khác). Kênh truyền song công (Full-duplex) được duy trì trực tiếp giữa hai máy.

2. Lập trình Socket TCP và Giao thức UDP Broadcast

- **TCP Socket:** Cung cấp kênh truyền tin cậy, bảo đảm đúng thứ tự gói tin qua bắt tay 3 bước (3-way handshake).
- **UDP Broadcast:** Phát gói tin định kỳ (PRESENCE_INTERVAL = 5s, PEER_TIMEOUT = 15s) đến địa chỉ **255.255.255.255:15000** để phát hiện các Peer lân cận.

3. Kỹ thuật Framing Length-Prefix

Do bản chất stream byte của TCP, dữ liệu được đóng gói dưới dạng: [4-byte Header độ dài Big-Endian] + [Dữ liệu JSON]. Hàm receive_exact() sẽ đọc chính xác đúng số byte yêu cầu, ngăn ngừa triệt để lỗi dính gói hoặc xé gói. MAX_PACKET_SIZE được cấu hình là 128 KB.

4. Mô hình Mã hóa Lai (Hybrid Cryptography)

- **RSA-2048:** Mã hóa bất đối xứng dùng trong pha Handshake để xác thực identity và trao đổi an toàn khóa phiên.
- **Fernet / AES-256-GCM:** Mã hóa đối xứng nhanh chóng cho nội dung tin nhắn thời gian thực và các đoạn chunk truyền file.

5. Mô hình Quản lý Độ tin cậy TOFU (Trust On First Use)

Quản lý Fingerprint (mã băm SHA-256 của Public Key PEM hiển thị dạng Hex chuẩn SSH). Lần đầu kết nối mặc định gán TRUSTED; các lần sau nếu Fingerprint khớp gán VERIFIED; nếu Fingerprint thay đổi sẽ phát cảnh báo MISMATCH.

KIẾN TRÚC VÀ CẤU TRÚC MÃ NGUỒN

Mã nguồn dự án tổ chức theo kiến trúc phân lớp sạch sẽ trong thư mục `src/`:

- `main.py`: Khởi động ứng dụng và kiểm tra cổng.
- `config.py`: Quản lý các hằng số mạng, timeout, kích thước packet, cấu hình GUI và logging.
- `controllers/controller.py`: Cầu nối trung gian giữa giao diện GUI và P2PNode.
- `network/`: Gồm `node.py` (TCP Socket), `discovery.py` (UDP Broadcast) và `transfer_manager.py` (File transfer).
- `message/protocol.py`: Đóng gói, serialize, deserialize và kiểm tra tính hợp lệ của packet JSON.
- `security/`: Mã hóa Fernet, envelope AES-GCM và RSA keypair.
- `identity/` & `trust/`: Sinh Peer ID, Fingerprint, quản lý TrustStore và TOFUEngine.
- `storage/`: Lưu vết lịch sử trò chuyện và danh bạ.
- `gui/`: Chứa các thành phần giao diện như Sidebar, Chatbox và Transfer Panel.

CHI TIẾT LUỒNG HOẠT ĐỘNG CÁC CHỨC NĂNG

1. Luồng Khởi động & UDP Discovery

1. `main.py` khởi tạo `ChatApp`.
2. Tạo `P2PNode` ngầm, bind TCP Server Socket tại cổng quy định.
3. Chạy `DiscoveryService` thực hiện broadcast UDP mỗi 5 giây.
4. Khi nhận packet UDP, cập nhật vào danh sách `discovered_peers`. Xóa khỏi danh sách nếu quá 15 giây không thấy heartbeat.

2. Luồng Thiết lập Kết nối P2P & Handshake

1. Người dùng chọn Peer hoặc nhập IP:Port -> gọi `connect_to_peer()`.
2. Thực hiện kết nối TCP Client với thuật toán Retry Exponential Backoff.
3. Gửi gói tin `HANDSHAKE` chứa Public Key RSA.
4. Bên nhận gửi `HANDSHAKE_ACK` -> Bên khởi tạo sinh khóa Fernet Session Key, mã hóa bằng RSA Public Key của đối phương và gửi `SESSION_KEY`.
5. Cả hai thiết lập trạng thái phiên `active`.

3. Luồng Gửi/Nhận Tin nhắn & Chống treo GUI

1. Người dùng gõ tin nhắn -> `Controller.send_message()`.
2. Mã hóa payload bằng Session Key -> Đóng gói Length-Prefix -> Gửi qua `socket.sendall()`.
3. Phía nhận: Luồng ngầm nhận byte -> Giải mã -> Lưu vào lịch sử.
4. Đẩy tin nhắn vào hàng đợi render. Sử dụng `self.after(0, self._drain_messages)` để render theo từng lô (batch=25), bảo đảm GUI không bị đóng băng.

4. Luồng Truyền tải Tập tin (File Transfer)

- 1) Người gửi kiểm tra file, tính băm SHA-256 -> Gửi `FILE_META`.
- 2) Phía nhận chọn "Download" -> Gửi `DOWNLOAD_REQUEST`

- 3) Bên gửi mở luồng truyền chia nhỏ file thành các chunk 64KB -> Gửi chuỗi **FILE_CHUNK**.
- 4) Bên nhận ghi dữ liệu tạm vào file **.part**. Khi nhận **FILE_COMPLETE**, kiểm tra mã băm SHA-256. Nếu khớp, đổi tên thành file chính thức.

THÀNH QUẢ ĐẠT ĐƯỢC

Bài đã hoàn thành 100% các yêu cầu đề ra, ứng dụng chạy ổn định và đáp ứng tốt các chức năng thực tế:

- ❖ Mạng P2P trực tiếp & Dò tìm tự động:
 - Kết nối trực tiếp giữa 2 máy qua Socket TCP mà không cần qua Server trung gian.
 - Tự động phát hiện các máy khác đang mở ứng dụng trong cùng mạng LAN bằng UDP Broadcast (Port 15000) cứ sau 5 giây.
- ❖ Chống lỗi truyền tin: Đã xử lý triệt để hiện tượng dính gói/xé gói trên TCP nhờ đóng gói Header 4-byte chỉ độ dài (Length-Prefix) và thuật toán `receive_exact()`.
- ❖ Đầy đủ tính năng Chat:
 - Nhắn tin văn bản thời gian thực với độ trễ gần như bằng 0.
 - Hỗ trợ trích dẫn (Reply) tin nhắn cũ, chuyển tiếp (Forward) tin nhắn cho máy khác.
 - Tích hợp bảng chọn Emoji và tự động gửi/hiển thị Avatar cá nhân.
- ❖ Truyền file an toàn:
 - Cho phép gửi file (PDF, Word, TXT, ảnh, ZIP...) dung lượng tối đa 10 MB.
 - Tự động chia nhỏ file thành các đoạn 64 KB để truyền, có thanh tiến trình hiển thị và kiểm tra mã băm SHA-256 để đảm bảo file nhận được không bị lỗi hay hư hỏng.
- ❖ Bảo mật & Quản lý tin cậy (TOFU):

- Bắt tay trao đổi khóa RSA-2048 và mã hóa tin nhắn bằng khóa phiên (Fernet / AES-256-GCM).
 - Tự động lưu vân tay (Fingerprint) của bạn chat. Nếu đối phương đổi máy hoặc có ai cố tình giả mạo, hệ thống sẽ báo cảnh báo (MISMATCH) ngay.
- ❖ Giao diện GUI mượt mà: Tách toàn bộ việc xử lý mạng sang các luồng ngầm (Background Threads) nên giao diện không bao giờ bị đơ hay treo (UI Freeze), kể cả khi nhận tin nhắn dồn dập hay đang tải file.

BẢNG PHÂN CÔNG CÔNG VIỆC CHI TIẾT

STT	Thành viên	Vai trò	Nội dung công việc đảm nhận
1	Huỳnh Văn Tại	Team Lead	Core Socket TCP, Server/Client lifecycle, kiến trúc hệ thống.
2	Ngô Đặng Minh Khôi	Developer	UDP Discovery, quản lý peer_registry, cấu hình config.
3	Trần Ngô Duy Khang	GUI Dev	Giao diện CustomTkinter, MainWindow, Sidebar, chuyển đổi UI.
4	Ngũ Hoàng Khang	Network Dev	JSON Protocol, Length-Prefix, Reply, Forward, Emoji, Avatar.
5	Nguyễn Thanh Khánh	File Dev	TransferManager, phân đoạn Chunk 64KB, kiểm tra SHA-256.
6	Lương Quốc Khánh	System Dev	IdentityManager, TOFUEngine, TrustStore, bộ Unit Test.

KHÓ KHĂN GẶP PHẢI

Là dự án đầu tay về Lập trình mạng và chưa từng có kinh nghiệm làm sản phẩm thực tế, nhóm sinh viên năm 2 đã gặp rất nhiều sai sót, lúng túng trong quá trình triển khai và phải tốn nhiều thời gian thử sai (trial-and-error) mới tìm ra giải pháp xử lý.

1. Sự cố dính gói và xé gói trên TCP Stream

- **Vấn đề thực tế:** Ban đầu do chưa hiểu bản chất dòng byte (byte-stream) của TCP, nhóm chỉ gọi hàm `recv(1024)` đơn giản. Hậu quả là khi đối phương gửi tin nhắn nhanh hoặc gửi file, gói tin liên tục bị "xé" ra thành nhiều mảnh rác hoặc bị "dính" gộp 2-3 tin nhắn JSON vào một lần đọc. Điều này làm hàm `json.loads()` bị văng lỗi `JSONDecodeError` liên tục và ứng dụng bị crash.
- **Cách khắc phục:** Nhóm phải đập đi viết lại phần giao thức, thêm Header 4-byte chỉ độ dài gói tin (Length-Prefix) ở đầu và viết riêng hàm `receive_exact()` để lặp cho đến khi đọc đủ chính xác số byte của gói tin đó mới xử lý tiếp.

2. Xung đột luồng làm đơ và crash giao diện Tkinter

- **Vấn đề thực tế:** Khi nhận tin nhắn từ luồng mạng (Background Thread), nhóm đã gọi trực tiếp các lệnh cập nhật giao diện Tkinter (như `.insert()`, `.config()`). Điều này làm lộ ra lỗi Thread-Safety nghiêm trọng: giao diện bị đóng băng (UI Freeze), hiển thị sai lệch nội dung, hoặc văng lỗi `Tcl_AsyncDelete` tắt ứng dụng đột ngột.
- **Cách khắc phục:** Nhóm phải tái cấu trúc toàn bộ luồng dữ liệu, cho luồng mạng đẩy tin nhắn vào một hàng đợi an toàn (deque) và dùng lệnh `self.after(0, ...)` để trả quyền vẽ giao diện về cho luồng chính của Tkinter xử lý theo từng lô (batch).

3. Băng thông bị nghẽn do chuỗi ảnh Avatar quá lớn

- **Vấn đề thực tế:** Lần đầu làm tính năng gửi Avatar, nhóm đem nguyên file ảnh gốc mã hóa sang chuỗi Base64 rồi nhét vào gói tin JSON Handshake. Chuỗi Base64 của một tấm ảnh dung lượng vài MB làm gói tin bị phình to khủng khiếp, vượt quá bộ nhớ đệm Socket làm gián đoạn quá trình bắt tay (Handshake) và gây trễ (lag) mạng rất nặng.

- **Cách khắc phục:** Nhóm nhận ra sai sót và bổ sung bước xử lý ảnh ở Client: bắt buộc nén và thu nhỏ kích thước (resize) Avatar về chuẩn 128x128 pixel trước khi mã hóa Base64 để gói tin luôn gọn nhẹ dưới 10 KB.

4. Dò tìm Peer (UDP Discovery) thất bại do Tường lửa và Wi-Fi

- **Vấn đề thực tế:** Khi đem ứng dụng đi test thực tế trên máy các thành viên trong nhóm, dịch vụ UDP Discovery hoạt động rất chậm chạp. Có máy không hề nhận được gói tin Broadcast nào, hoặc 2 máy cùng bắt chung một mạng Wi-Fi ở quán cà phê nhưng hoàn toàn không "thấy" nhau.
- **Nguyên nhân & Xử lý:** Nhóm mất nhiều thời gian mới phát hiện ra Tường lửa (Windows Firewall) mặc định đã chặn các gói tin UDP ngầm, đồng thời một số router Wi-Fi bật tính năng AP Isolation (chặn các thiết bị trong cùng Wi-Fi giao tiếp với nhau). Nhóm phải thêm hướng dẫn người dùng cấp quyền Firewall thủ công và bổ sung ô cho phép nhập trực tiếp IP:Port để kết nối bằng tay phòng khi mạng bị chặn Broadcast.

KIẾN THỨC / KỸ NĂNG HỌC ĐƯỢC

- Thấu hiểu bản chất lập trình Socket mạng cấp thấp và các giao thức TCP/UDP.
- Làm chủ kỹ thuật lập trình đa luồng (Multithreading) và xử lý đồng bộ.
- Áp dụng mô hình Mã hóa lai (Hybrid Cryptography) và xác thực TOFU trong thực tế.
- Nâng cao kỹ năng làm việc nhóm và quản lý mã nguồn trên GitHub.

NỘI DUNG ĐÃ VÀ CHƯA HOÀN THÀNH

1. Nội dung đã hoàn thành (100%)

- Mạng P2P kết nối trực tiếp qua TCP Socket và tự động dò tìm qua UDP Broadcast.
- Đóng gói JSON Packet với Length-Prefix 4-byte.
- Trao đổi khóa RSA, mã hóa phiên bằng Fernet/AES-256-GCM.
- Quản lý định danh Identity và độ tin cậy TOFU.
- Nhắn tin trực tiếp, Reply, Forward, hiển thị Avatar và gửi Emoji.
- Truyền tập tin phân đoạn 64KB có checksum SHA-256.

2. Nội dung chưa hoàn thành / Hạn chế

- Chưa triển khai cơ chế NAT Traversal (STUN/TURN) để kết nối ngoài mạng LAN.
- Chưa mã hóa cơ sở dữ liệu lịch sử chat lưu trữ cục bộ trên đĩa cứng.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đồ án "Ứng dụng Chat P2P không dùng Server trung tâm (UDM09)" đã hoàn thành trọn vẹn các mục tiêu cốt lõi đề ra. Thông qua việc tự tay thiết kế và triển khai một ứng dụng mạng ngang hàng từ con số 0, nhóm sinh viên năm 2 đã đạt được những kết quả quan trọng:

1. Về mặt sản phẩm: Ứng dụng xây dựng thành công bộ xương sống kết nối P2P thuần túy qua TCP Socket, hỗ trợ tự động tìm kiếm máy lân cận bằng UDP Broadcast trong mạng LAN. Hệ thống tích hợp đầy đủ các tính năng thực tế như chat thời gian thực, trích dẫn (Reply), chuyển tiếp (Forward), gửi Emoji, hiển thị Avatar mã hóa Base64 và truyền file phân đoạn (Chunking 64KB) có kiểm tra mã băm SHA-256.
2. Về mặt kỹ thuật & Bảo mật: Nhóm đã xử lý triệt để bài toán dính/xé gói tin TCP nhờ giao thức Length-Prefix, không chế hoàn toàn lỗi đơ/treo giao diện (UI Freeze) bằng mô hình đa luồng kết hợp hàng đợi bất đồng bộ, đồng thời bảo vệ dữ liệu truyền đi qua cơ chế mã hóa lai (RSA-2048 + Fernet/AES-256-GCM) và mô hình quản lý độ tin cậy TOFU.
3. Về mặt trải nghiệm học tập: Đồ án là một bước tiến lớn giúp nhóm bước ra khỏi lý thuyết hàn lâm, trực tiếp "va chạm" với các sự cố mạng thực tế (xung đột luồng, tường lửa chặn gói tin, nghẽn bộ nhớ đệm Socket). Qua đó, nhóm đã tích lũy được tư duy lập trình hệ thống, kỹ năng debug đa luồng và quy trình làm việc nhóm bài bản trên Git/GitHub.

Hạn chế còn tồn tại

Bên cạnh các kết quả đạt được, do hạn chế về thời gian và kinh nghiệm làm dự án thực tế lần đầu, ứng dụng vẫn còn một số điểm chưa tối ưu:

- Phạm vi kết nối bị giới hạn trong mạng LAN: Ứng dụng mới chỉ hoạt động được khi các máy kết nối chung một mạng nội bộ hoặc có IP định tuyến trực tiếp. Chưa

có giải pháp để 2 máy ở 2 mạng Wi-Fi khác nhau (qua Internet) tìm thấy và chat với nhau.

- Lịch sử chat lưu đĩa chưa mã hóa: File lưu lịch sử tin nhắn và danh bạ cục bộ (message_history.py) hiện tại vẫn lưu dưới dạng dữ liệu thô (JSON/Text) trên đĩa cứng, chưa được mã hóa file để chống đọc lén từ chính thiết bị đó.
- Chưa hỗ trợ truyền file dung lượng cực lớn: Tính năng truyền file hiện mới chỉ dừng lại ở giới hạn tối đa 10 MB để đảm bảo bộ nhớ đệm; chưa có cơ chế tạm dừng/tiếp tục (Pause/Resume) khi gặp sự cố đứt gãy mạng đột ngột.

Hướng phát triển trong tương lai

Nhằm hoàn thiện ứng dụng thành một giải pháp truyền thông mạng ngang hàng toàn diện hơn, nhóm đề ra các hướng nâng cấp tiếp theo:

1. Mở rộng kết nối xuyên Internet (NAT Traversal):
2. Nâng cấp bảo mật dữ liệu lưu trữ cục bộ (Local Storage Security):
 - Chuyển đổi cơ chế lưu file JSON thô sang cơ sở dữ liệu như SQLite / SQLCipher.
 - Toàn bộ lịch sử trò chuyện lưu trữ dưới đĩa cứng sẽ được mã hóa bằng chìa khóa cá nhân (Master Passphrase) của người dùng.
3. Hoàn thiện tính năng Truyền file dung lượng lớn (Large File Streaming):
 - Cải tiến module TransferManager để hỗ trợ truyền file dung lượng hàng trăm MB đến GB.
 - Bổ sung tính năng tự động ghi nhớ vị trí chunk đã nhận (Offset checkpointing) để cho phép Pause / Resume truyền file khi mạng chập chờn.
4. Mở rộng Gọi thoại và Video P2P (Voice / Video Call):
 - Tận dụng hạ tầng Socket TCP/UDP sẵn có để tích hợp thêm thư viện xử lý luồng âm thanh/hình ảnh thời gian thực (như WebRTC hoặc OpenCV/PyAudio).

THÔNG TIN REPOSITORY

1. Toàn bộ mã nguồn, Unit Tests và tài liệu kỹ thuật được nhóm công khai trên Link GitHub repository:

https://github.com/khangtn3072-lang/UDM_9_CHAT-P2P

2. Cách chạy thử (Windows):

```
pip install customtkinter cryptography PyJWT pywintypes
```

```
python Code/P2PChat/src/main.py
```

3. Video báo cáo và demo ứng dụng.

<https://youtu.be/xPmyHIWyuEU>